

VS-Movies Group (vsmovies)

You just joined a group of fans of the *vs-movies*, the movies whose plot is the battle of two superheroes. Each of the N members of this group has to propose his favourite movie of this genre; the i -th member chooses A_i vs B_i , a T_i minutes long movie.

This night the members will watch some of the movies, following the strict rules of the group:

- The first member will present his movie, telling the story of the two superheroes in it.
- Then all the members will watch that movie.
- After the movie ends another member proposes his chosen movie, with the only rule that the story of exactly one of the two superheroes in it has already been told.
- After explaining the story of the other hero, all the members will watch that movie.
- This process is repeated until all the S superheroes are present in at least a movie.

Selecting the order in which the movies are watched is not an easy task, especially since every member wishes to show his favourite movie to the other members. To avoid any conflicts all the members share their favourite movie with each other and each member writes his proposal list: the list of movies to watch, which of course will include his favourite movie.

To maximise the probability of being selected, each list should be as short as possible; that is, the sum of the duration of the selected movies should be minimized. Of course, all the S superheroes must still be present in each proposal list.

You want to help to speed up this process by writing a program that, given the list of favourite movies, finds the duration of the optimal plan for each member.

 Among the attachments of this task you may find a template file `vsmovies.*` with a sample incomplete implementation.



Input

The first line contains the integers N and S , the number of members of the group and the total number of superheroes. The following N lines contain 3 integers each: A_i , B_i , T_i , the description of the favourite movie of the i -th member.

Output

You need to write N lines with an integer each: the duration of the shortest plan that contains the i -th movie and involves all the superheroes.

Constraints

- $1 \leq N \leq 500\,000$.
- $2 \leq S \leq 100\,000$.
- $0 \leq A_i, B_i \leq S - 1$ for each $i = 0 \dots N - 1$.
- $A_i \neq B_i$ for each $i = 0 \dots N - 1$.
- $1 \leq T_i \leq 10^9$ for each $i = 0 \dots N - 1$.
- There could be more than one movie involving the same pair of heroes.
- It's always possible to find a plan.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

- **Subtask 1** (0 points) Examples.

- **Subtask 2** (5 points) $S = 2$.

- **Subtask 3** (12 points) $S = 3$.

- **Subtask 4** (14 points) $N \leq 15$.

- **Subtask 5** (22 points) $N \leq 1000$.

- **Subtask 6** (26 points) $N \leq 50\,000$ and $S \leq 50\,000$.

- **Subtask 7** (21 points) No additional limitations.


Examples

input	output
4 3 0 1 10 0 1 5 0 2 2 1 2 4	12 7 6 6
3 4 0 1 1 0 2 2 0 3 3	6 6 6

Explanation

In the **first sample case**, each member presents the following lists:

- the first member's list contains the first and third movies, with total duration of 12;
- the second member's list contains the second and third movies, with total duration of 7;
- the third member's list contains the third and fourth movies, with total duration of 6;
- the fourth member's list is the same list of the third member.

In the **second sample case**, all the three movies must be present in each one of the proposed lists. This means that each member presents the same list, whose total duration is 6.